ELAANJI — Premium Garments & Saree E-Commerce Platform

Comprehensive Project Documentation

1. Project Overview

ELAANJI is a state-of-the-art, full-stack e-commerce web application engineered specifically for a high-end luxury garments and traditional saree brand. The platform features an ultra-responsive, mobile-first storefront for customers and a feature-rich Admin Backoffice Portal for store operations, inventory management, order processing, and analytics.

Key Highlights

- **Modern Architecture**: Decoupled RESTful API architecture powered by FastAPI (Python) and React 18 (Vite).
- **Authentication**: JWT token-based authentication with Access/Refresh token rotation, plus native **Google OAuth 2.0** Sign-In integration.
- **Role-Based Access Control (RBAC)**: Enforced middleware security differentiating Customer and Admin routes.
- **Media & CDN**: Image upload and optimization pipeline integrated with **Cloudinary**.
- **Payment Processing**: Integrated with **Razorpay** payment gateway.
- **Database & Migrations**: PostgreSQL database schema managed via **SQLAlchemy 2.0 ORM** and **Alembic** migrations.
- **CI/CD Pipeline**: GitHub Actions workflow for automated code compilation, linting, and production build checks.
- **Production Deployment**: Deployment ready with Render (Backend API + PostgreSQL) and Vercel (Frontend SPA).

2. Technology Stack

Backend Stack

- **Framework**: FastAPI (Python 3.12)
- **Database**: PostgreSQL
- **ORM & Migrations**: SQLAlchemy 2.0 & Alembic
- **Security & Auth**: PyJWT / Python-Jose, Passlib with Bcrypt password hashing
- **OAuth Integration**: Google Auth library (`google-auth`)
- **Storage & CDN**: Cloudinary Python SDK
- **Payment Gateway**: Razorpay Python SDK
- **Rate Limiting**: SlowAPI (IP & endpoint rate limiting)
- **Application Server**: Uvicorn (ASGI server)

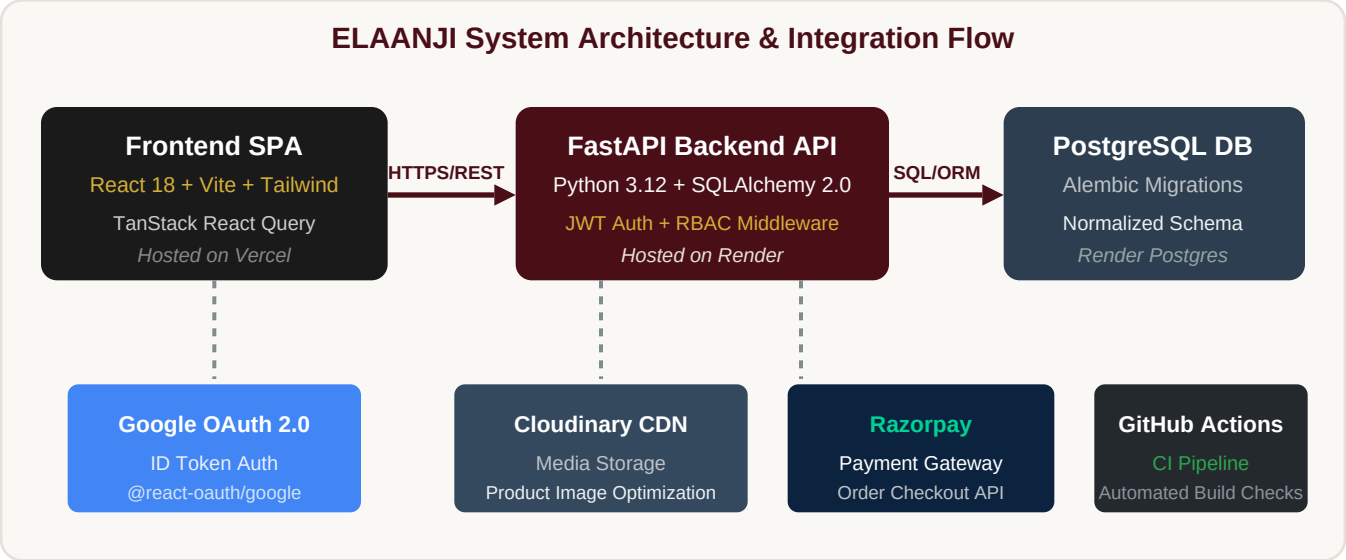
Frontend Stack

- **Framework**: React 18 with Vite (ES Modules)

- **Styling & Design System**: Tailwind CSS (Custom luxury color palette, typographic hierarchy with Outfit & Playfair Display fonts)
- **State & Data Fetching**: TanStack React Query (v5) & Axios
- **Routing**: React Router DOM (v6)
- **Icons & Motion**: Lucide React & Framer Motion
- **Analytics Charts**: Recharts
- **OAuth Component**: `@react-oauth/google`

---

3. System Architecture & Project Structure




```
elaanji/
■■■ .github/
■ ■■■ workflows/
■ ■■■ ci.yml # GitHub Actions CI workflow (Backend & Frontend checks)
■■■ backend/
■ ■■■ app/
■ ■ ■■■ api/ # REST API Routers
■ ■ ■ ■■■ admin/ # Admin Endpoints (Products, Orders, Inventory, Categories, etc.)
■ ■ ■ ■■■ auth.py # Registration, Login, Google OAuth, Refresh, Password Reset
■ ■ ■ ■■■ products.py # Public Product Catalog APIs
■ ■ ■ ■■■ cart.py # Shopping Cart APIs
■ ■ ■ ■■■ orders.py # Checkout & Order Placement APIs
■ ■ ■ ■■■ ...
■ ■ ■■■ core/ # Configuration, Database, Security, Rate Limiter
■ ■ ■ ■■■ config.py # Centralized Pydantic Settings
■ ■ ■ ■■■ database.py # SQLAlchemy Engine & Session
■ ■ ■ ■■■ security.py # JWT generation & password hashing
■ ■ ■■■ middleware/ # Auth & RBAC Middleware
■ ■ ■■■ models/ # SQLAlchemy Database Models (User, Product, Order, etc.)
■ ■ ■■■ schemas/ # Pydantic Request/Response Models
■ ■ ■■■ services/ # Business Logic & External Integrations
■ ■■■ migrations/ # Alembic Database Migration Scripts
■ ■■■ scripts/ # CLI Tools (create_admin.py, seed_catalog.py)
■ ■■■ requirements.txt # Python Dependencies
■ ■■■ render.yaml # Render Infrastructure Definition
```

```
■■■ frontend/
■■■ src/
■ ■■■ admin/ # Admin Backoffice Portal Pages & Components
■ ■■■ api/ # Axios API Clients & Service Layer
■ ■■■ components/ # Reusable UI Components (Navbar, Footer, ProductCard, etc.)
■ ■■■ context/ # React Contexts (AuthContext, ToastContext)
■ ■■■ pages/ # Public Storefront Pages (Home, Shop, ProductDetail, Cart, Checkout, Account)
■ ■■■ App.jsx # Router & Application Root
■■■ package.json # NPM Dependencies & Build Scripts
■■■ vite.config.js # Vite Configuration & Proxy Setup
```

4. Database Schema & Data Models

The database schema is designed with high normalization, transactional safety, and foreign key cascades.

Core Entities:

2. Category: Supports hierarchical subcategories (`parent_id`), display ordering, and safe unassignment on deletion.

3. Brand: Represents clothing sub-brands / collections.

4. Product: Catalog items containing title, slug, SKU, price, discount, saree specs (fabric, border type, work type, saree length), status, tags, and rating summaries.

5. ProductVariant: Specific purchasable SKU combinations (Color, Blouse Size, Price Override, Stock Quantity, Low Stock Threshold).

6. ProductImage: Multi-image gallery with hover image designation and display order.

7. Cart & CartItem: Persistent server-side shopping cart.

8. Order & OrderItem: Transaction record capturing line items, snapshot prices, shipping address, payment status, tracking number, and order lifecycle states.

9. InventoryTransaction: Audit log for every stock increase, decrease, or adjustment.

10. ActivityLog: Admin audit trail logging every backoffice mutation.

5. Functional Features & Capabilities

■■ Customer Storefront

• **Responsive Navigation**: Minimalist luxury header, megamenu navigation, live cart count, and quick search.

• **Product Catalog & Filters**:

• Filter by Category / Subcategory

• Filter by Saree Tradition (Kanjivaram, Banarasi, Cotton, Silk, Organza, etc.)

• Filter by Occasion (Wedding, Festive, Party, Casual)

• Price range & sorting (Newest, Price Low-to-High, Price High-to-Low)

• **Product Details Page**:

• High-resolution image gallery with interactive thumbnail switching

• Dynamic variant picker (Color & Blouse Size selection)

- Live stock status badge (In Stock, Low Stock, Out of Stock)
- Accordion sections for Fabric details, Care Instructions, & Shipping info
- Verified Customer Reviews & Rating distribution
- **Authentication**:
- Standard Email + Password registration & login
- Single-click **Google OAuth 2.0 Sign-In**
- Session persistence via JWT token refresh
- **Checkout & Orders**:
- Address book management (Save multiple addresses with default selection)
- Coupon code validation at checkout
- Order summary breakdown (Subtotal, Tax, Shipping Fee calculation)
- Order History & Status Tracking page

■■ Admin Backoffice Portal (`/admin`)

- **Dashboard & Analytics**: Real-time sales KPIs, order status breakdown charts (Recharts), and low-stock alert widgets.
- **Product Management**:
- Add, Edit, Duplicate, & Archive products
- Manage variants (colors, sizes, stock levels)
- Direct Cloudinary drag-and-drop image upload
- **Category Management**:
- Hierarchy management (Categories & Subcategories)
- Display reordering
- Safe category deletion (automatically unassigns linked products without deleting catalog data)
- **Inventory Control**:
- Real-time stock audit trail logging every stock adjustment
- Low-stock filter view
- **Order Management**:
- Filter orders by status (`pending`, `processing`, `shipped`, `delivered`, `cancelled`)
- Update fulfillment status and add shipping carrier tracking numbers
- **Banner & Content Management**:
- Upload & manage homepage promo banners with custom CTA links
- **Customer & Review Moderation**:
- Manage registered customer accounts
- Approve or reject submitted product reviews
- **Admin Utilities**:

- Activity Log audit trail
- CLI Admin creation tool (`scripts/create_admin.py`)

---

6. Environment Variables Reference

Backend `.env` (`backend/.env`)

Variable	Description	Example
`DATABASE_URL`	PostgreSQL connection string	`postgresql+psycopg2://postgres:pass@localhost:5432/garments_db`
`JWT_SECRET_KEY`	Secret key for signing JWT tokens	`super-secret-key-32-chars-long`
`JWT_ALGORITHM`	Algorithm for JWT tokens	`HS256`
`ACCESS_TOKEN_EXPIRE_MINUTES`	Access token lifespan in minutes	`60`
`FRONTEND_ORIGIN`	Allowed CORS origin	`http://localhost:5173`
`CLOUDINARY_CLOUD_NAME`	Cloudinary cloud identifier	`your_cloud_name`
`CLOUDINARY_API_KEY`	Cloudinary API Key	`1234567890`
`CLOUDINARY_API_SECRET`	Cloudinary API Secret	`your_cloudinary_secret`
`GOOGLE_CLIENT_ID`	Google OAuth Client ID for backend verification	`xxxx.apps.googleusercontent.com`
`RAZORPAY_KEY_ID`	Razorpay Key ID	`rzp_test_xxxx`
`RAZORPAY_KEY_SECRET`	Razorpay Key Secret	`your_razorpay_secret`

Frontend `.env` (`frontend/.env`)

Variable	Description	Example
`VITE_GOOGLE_CLIENT_ID`	Google OAuth Client ID for frontend component	`xxxx.apps.googleusercontent.com`

---

7. Local Development Setup Guide

Prerequisites

- Python 3.12+
- Node.js 20+
- PostgreSQL database running locally or in cloud

Step 1: Backend Setup

```
cd backend

# 1. Create virtual environment
python -m venv venv

# 2. Activate virtual environment
# Windows (PowerShell):
.\venv\Scripts\Activate.ps1
# macOS/Linux:
source venv/bin/activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Run database migrations
python -m alembic upgrade head

# 5. Seed initial product catalog (optional)
python scripts/seed_catalog.py

# 6. Create initial admin user
python scripts/create_admin.py admin@elaanji.com MyPassword123! "Super Admin"

# 7. Start backend server
uvicorn app.main:app --reload --port 8000
```

Backend API interactive documentation will be available at: **`http://localhost:8000/docs`**

Step 2: Frontend Setup

```
cd frontend

# 1. Install dependencies
npm install

# 2. Start Vite dev server
npm run dev
```

Frontend Web Application will be available at: **`http://localhost:5173`**

---

8. Deployment Architecture (Render + Vercel)

Backend Deployment (Render)

- **Environment**: Render Web Service (Python Runtime)
- **Build Command**: **`pip install -r requirements.txt && alembic upgrade head`**
- **Start Command**: **`uvicorn app.main:app --host 0.0.0.0 --port $PORT`**
- **Database**: Render PostgreSQL Database

Frontend Deployment (Vercel)

- **Framework Preset**: Vite
- **Build Command**: **`npm run build`**
- **Output Directory**: **`dist`**

- **Rewrites (`vercel.json`)**: Configured for SPA single-page routing (`/* -> /index.html`)

---

9. Testing & CI/CD Pipeline

The project includes an automated GitHub Actions workflow defined in `.github/workflows/ci.yml`.

Automated Pipeline Jobs:

1. Backend CI:

- Sets up Python 3.12 environment.
- Installs all dependencies from `requirements.txt`.
- Compiles Python files (`compileall`) to verify syntax integrity across all routers, services, and models.

2. Frontend CI:

- Sets up Node.js 20.
- Executes `npm ci` clean install.
- Executes `npm run build` to verify production Vite bundle compilation without asset or import errors.

---

10. Security & Quality Features

- **Password Hashing**: Passlib with Bcrypt algorithm.
- **Rate Limiting**: Endpoint protection on authentication routes (`/api/auth/register`, `/api/auth/login`, `/api/auth/google`) using SlowAPI limiter.
- **JWT Protection**: Tokens contain payload claims and expiration times; invalid or tampered tokens are rejected with HTTP 401.
- **Safe Resource Cleanup**: Database deletions for categories, products, images, and users protect database integrity using explicit foreign key constraints and unassignment routines.